

Chapitre V: Javascript

Professeure Manal ZETTAM

Ecole Nationale des Sciences Appliquées de Safi

Université Cadi Ayyad - Marrakech

Année Universitaire : 2024-2025

Plan

- 1 introduction
- 2 Inclusion du code JavaScript
- 3 Manipulez des données avec JavaScript
 - Déclarer et afficher une variable
 - Déclarer et afficher une constante
 - Les types de données et les opérations

Plan

4 Les objets en JavaScript

- Déclarer un objet en JavaScript
- Accédez à une propriété d'un objet JavaScript
- Modifier la valeur des propriétés d'un objet JavaScript

5 Les tableaux en JavaScript

- Déclarer un tableau en JavaScript
- Accéder aux éléments du tableau
- La propriété length et les méthodes push(), pop() et splice()

Plan

6 Les conditions et les boucles

- Instruction if ... else
- Instruction switch
- Opérateur ternaire
- Boucle for
- Instruction while

7 Les fonctions

Plan

- 8 Elements HTML et javascript
 - Récupérer un élément d'une page web
 - Modifiez un élément d'une page web
 - Ajouter un nouvel élément dans une page web
 - Evénements et interactions

- 9 Bibliographie & Webographie

Introduction

JavaScript né en 1995, est un langage de script côté client, ce qui signifie qu'il s'exécute côté client, dans un navigateur Web.

Inclusion du code JavaScript

Le code javascript peut être écrit:

- ① dans un fichier .js (recommandé) ;
- ② dans l'en-tête <head> ou le corps <body> du fichier HTML ;
- ③ directement dans les balises du fichier HTML via des pseudo-URL(href) ou des attributs d'événements tels que onload (déconseillé).

Inclusion du code JavaScript

Code javascript dans l'en-tête <head> ou le corps <body> du fichier HTML

```
<body>  
<script language="JavaScript" type="text/JavaScript">  
window.alert("Welcome to JavaScript!");  
</script>  
</body>
```

Inclusion du code JavaScript

Code javascript directement dans les balises du fichier HTML

```
<a href="#" onclick="window.alert('Welcome to  
JavaScript!'); return false;">click here</a>
```

Inclusion du code JavaScript

Code javascript directement dans les balises du fichier HTML

```
<a href="JavaScript:window.alert('Welcome to JavaScript!');  
>click here for a surprise</a>
```

Inclusion du code JavaScript

Code javascript dans un fichier .js

```
window.alert("Welcome to JavaScript!");
```

Inclusion du code JavaScript

les attributs d'événements

```
https://html.spec.whatwg.org/multipage/indices.html#  
event-pageshow
```

Déclarer et afficher une variable

L'instruction let: l'instruction let permet de déclarer une variable.

```
let mavariable = 42
```

L'instruction `console.log()`: permet d'afficher le contenu de la variable `mavariable` dans la console.

```
let mavariable = 42  
console.log(mavariable)
```

Déclarer et afficher une variable

Créer un fichier javascript avec l'extension .js:

```
let mavariable = 42  
console.log(mavariable)
```

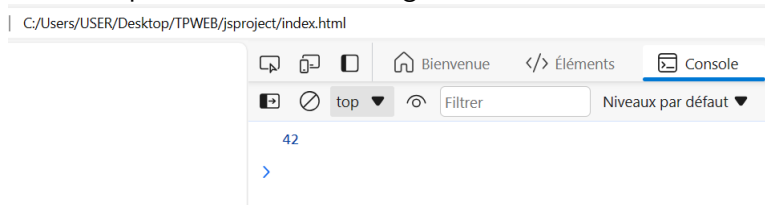
Déclarer et afficher une variable

Créer un fichier html et ajouter le code suivant

```
<!DOCTYPE html>
<html lang="fr">
  <body>
    <script src="script.js"></script>
  </body>
</html>
```

Déclarer et afficher une variable

ouvrir la page html avec un navigateur web. faire un clic droit puis choisir inspecter et chercher l'onglet console



Déclarer et afficher une variable

L'instruction var: l'instruction var permet de déclarer une variable.

```
var mavariable = 42
```

L'instruction `console.log()`: permet d'afficher le contenu de la variable `mavariable` dans la console.

```
var mavariable = 42  
console.log(mavariable)
```

Déclarer et afficher une variable

let vs var

La portée d'une variable déclarée via let est celle du bloc courant contrairement à une variable déclarée via var.

Déclarer et afficher une variable

Exemple

```
let x = 1;
if (x === 1) {
  let x = 2;

  console.log(x);
  // Expected output: 2
}

console.log(x);
// Expected output: 1
```

Déclarer et afficher une variable

Exemple

```
var x = 1;
if (x === 1) {
    var x = 2;

    console.log(x);
    // Expected output: 2
}

console.log(x);
// Expected output: 2
```

Déclarer et afficher une constante

L'instruction const: l'instruction const permet de déclarer une constante.

```
const monPrenom = "Manal"
```

L'instruction `console.log()`: permet d'afficher le contenu de la constante `monPrenom` dans la console.

```
const monPrenom = "Manal"  
console.log(monPrenom)
```

Déclarer et afficher une constante

Modifier le contenu du fichier javascript comme suit et rafraîchir la page web.

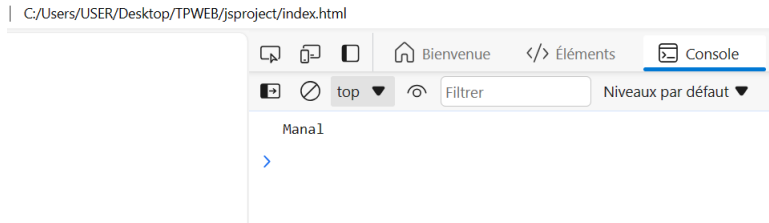
```
const monPrenom = "Manal"  
console.log(monPrenom)
```

ou

```
const monPrenom = "Manal" ;  
console.log(monPrenom);
```

Déclarer et afficher une constante

Affichage du contenu de la constante monPrenom dans la console



Les types de données et les opérations

En javascript, il existe sept types primitifs dont les trois suivants:

- **string** : correspond à une chaîne de caractères.
- **number** : correspond à un chiffre.
- **boolean** : correspond à un booléen.

`https:`

`//developer.mozilla.org/en-US/docs/Glossary/Primitive`

Les types de données et les opérations

Le type number

Pour le type number on a les opérateurs suivants:

- L'addition +,
- la soustraction - ,
- la multiplications *,
- la division /,

Les types de données et les opérations

Addition (+)

Syntaxe

L'addition de deux nombre x et y est notée comme suit sous javascript:

```
x + y; // addition de x et y
```

Les types de données et les opérations

Exemple

```
// Number + Number -> addition  
1 + 2; // 3  
  
// Boolean + Number -> addition  
true + 1; // 2  
  
// Boolean + Boolean -> addition  
false + false; // 0
```

Les types de données et les opérations

La concaténation: la concaténation de deux chaines de caractères est effectuée via l'opérateur `+`. **Exemple**

```
let premierePartie = "Mon nom est Olivia"  
let deuxiemePartie = ", Olivia Gazalée."  
let punchline = premierePartie + deuxiemePartie  
// punchline vaut \Mon nom est Olivia, Olivia Gazalée."
```

Les types de données et les opérations

Soustraction (-)

Syntaxe

La soustraction de deux nombre x et y est notée comme suit sous javascript:

```
x - y; // soustraction de x et y
```

Les types de données et les opérations

Exemple

```
5 - 3; // 2
3 - 5; // -2
```

Les types de données et les opérations

documentation

La syntaxe et les exemples sur les autres opérateurs sont disponible sur la page web <https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Operators>.

Les objets en JavaScript

Objet

Un objet en JavaScript est un conteneur. Il est composé de propriétés/attributs qui ont chacune une valeur.

Déclarer un objet en JavaScript

En Javascript, on déclare un objet comme suit:

```
let nom_objet = {  
  propriete_1: valeur_1,  
  propriete_2: valeur_2,  
  propriete_1: valeur_3,  
}
```

Déclarer un objet en JavaScript

Exemple

```
let Client = {  
  nom: "Benis",  
  prenom: "Meryem",  
  tel: 212666666666,  
  ville: "safi"  
}
```

Accédez à une propriété d'un objet JavaScript

Pour accéder à la valeur d'une propriété, il suffit d'utiliser le point .

```
console.log(nom_objet.propriete_1);
```

Accédez à une propriété d'un objet JavaScript

Exemple

```
console.log(Client.nom);
```

Modifier la valeur des propriétés d'un objet JavaScript

Pour modifier la valeur d'une propriété il suffit d'utiliser le point . pour accéder à la propriété et puis l'opérateur d'affectation = comme suit:

```
nom_objet.propriete_1=valeur_k;
```

Modifier la valeur des propriétés d'un objet JavaScript

Exemple

```
console.log(Client.nom);  
Client.nom = "Obaa";  
console.log("le nouveau nom est "+Client.nom);
```

Les tableaux en JavaScript

Les tableaux sont généralement décrits comme des "objets de type liste" ; un tableau est un objet contenant plusieurs valeurs.

Déclarer un tableau en JavaScript

Pour déclarer un tableau en JavaScript, il suffit d'utiliser les crochets [] . Chaque valeur contenue dans le tableau est séparée par une virgule , :

```
let nom_tableau = [valeur_0, valeur_1, ..., valeur_n-1];
```

Les tableaux en JavaScript

Exemple

```
let shopping = ["pain", "lait", "fromage", "houmous", "nouilles"];
let sequence = [1, 1, 2, 3, 5, 8, 13];
let random = ["arbre", 795, [0, 1, 2]];
console.log(shopping);
console.log(sequence);
console.log(random);
```

Accéder aux éléments du tableau

Pour accéder à un élément du tableau, il suffit d'écrire le nom du tableau suivi l'index de l'élément souhaité entre crochets comme suit:

```
let nom_tableau = [valeur_0, valeur_1, ..., valeur_n-1];  
let kiemeelement = nom_tableau[k]
```

Accéder aux éléments du tableau

Exemple

```
let shopping = ["pain", "lait", "fromage", "houmous", "nouilles"];  
let premierelement = shopping[0];  
console.log(premierelement);  
shopping[0]="modifier";  
console.log(shopping);
```

La propriété `length` et les méthodes `push()`, `pop()` et `splice()`

`length`

La propriété **`length`** permet de connaître le nombre d'éléments que contient un tableau.

Exemple

```
let shopping = ["pain", "lait", "fromage", "houmous", "nouilles"];
const nombreDeShopping = shopping.length
console.log(nombreDeShopping )
// nombreDeShopping vaut 5
```

La propriété length et les méthodes push() ,pop() et splice()

push()

Pour ajouter des données à un tableau, il suffit d'utiliser la méthode **push** qui prend en paramètre la valeur à ajouter.

Exemple

```
let shopping = ["pain", "lait", "fromage", "houmous", "nouilles"];
shopping.push("poisson")
console.log(shopping)
// shopping vaut ["pain", "lait", "fromage", "houmous", "nouilles",
// "poisson"]
```

La propriété length et les méthodes push() ,pop() et splice()

pop()

Pour supprimer le dernier élément du tableau, il suffit d'utiliser la méthode pop.

Exemple

```
let shopping = ["pain", "lait", "fromage", "houmous", "nouilles"];
shopping.pop()
console.log(shopping)
// shopping vaut ["pain", "lait", "fromage", "houmous"]
```

La propriété length et les méthodes push() ,pop() et splice()

splice()

La méthode splice() modifie le contenu d'un tableau en retirant des éléments et/ou en ajoutant de nouveaux éléments.

Exemple

```
let shopping = ["pain", "lait", "fromage", "houmous", "nouilles"];
shopping.(1, 0, 'poisson');
console.log(shopping)
// shopping vaut ["pain", "poisson", "lait", "fromage", "houmous"]
```

La propriété length et les méthodes push() ,pop() et splice()

Documentation Array

https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Global_Objects/Array



L'opérateur d'affectation permet seulement de faire des copies par références et non pas des copies par valeurs dans le cas d'un objet tel un tableau.

Instruction if ... else

Syntaxe

```
if (condition) {  
    code à exécuter si la condition est true  
} else {  
    sinon exécuter cet autre code à la place  
}
```

Instruction if ... else

Exemple

```
let coursesFaites = false;

if (coursesFaites === true) {
  let argentDePoche = 10;
} else {
  let argentDePoche = 5;
}

console.log(argentDePoche)
// argentDePoche vaut 5
```

Instruction if ... else

Opérateurs logiques AND, OR et NOT

Pour tester plusieurs conditions sans imbriquer des instructions if...else , il suffit d'utiliser les opérateurs logiques tels que:

- && — AND : permet d'enchaîner deux ou plusieurs expressions de sorte que toutes doivent être individuellement égales à true pour que l'ensemble de l'expression retourne true.
- || — OR : permet d'enchaîner deux ou plusieurs expressions ensemble de sorte qu'il suffit qu'une au plus soit évaluée comme étant true pour que l'ensemble de l'expression renvoie true.

Instruction if ... else

Exemple

```
let camionDeGlaces = true
let etatDeLaMaison ="on fire"
if (!(camionDeGlaces || etatDeLaMaison === "on fire")) {
  console.log("Vous pouvez probablement rester dedans.");
} else {
  console.log("Vous devriez sortir de la maison rapidement.");
}
```

Instruction switch

Syntaxe

```
switch (expression) {  
  case choix1:  
    exécuter ce code  
    break;  
  
  case choix2:  
    exécuter ce code à la place  
    break;  
  
  // incorporez autant de case que vous le souhaitez  
  
  default:  
    sinon, exécutez juste ce code  
}
```

Instruction switch

Exemple

```
<label for="weather">Select the weather type today: </label>
<select id="weather">
  <option value="">--Make a choice--</option>
  <option value="sunny">Sunny</option>
  <option value="rainy">Rainy</option>
</select>
<p></p>
```

Instruction switch

Exemple

```
const select = document.querySelector("select");
const para = document.querySelector("p");
select.addEventListener("change", setWeather);

function setWeather() {
  let choice = select.value;
  switch (choice) {
    case "sunny":
      para.textContent =
        "It is nice and sunny outside today. Wear shorts!
        Go to the beach, or the park, and get an ice cream.";
      break;
```

Instruction switch

Exemple

```
case "rainy":  
    para.textContent =  
        "Rain is falling outside; take a rain coat and a brolly,  
        and don't stay out for too long.";  
    break;  
default:  
    para.textContent = "";  
}  
}
```

Opérateur ternaire

L'opérateur ternaire ou conditionnel est un petit morceau de code qui teste une condition et renvoie une valeur ou expression si elle est true et une autre si elle est false.

Syntaxe

```
( condition ) ? exécuter ce code : exécuter celui-ci à la place
```

Opérateur ternaire

Exemple

```
let formuleDePolitesse = est_anniversaire
  ? "Bon anniversaire Mme Smith &nbsp;!"
  : "Bonjour Mme Smith.";
```

Boucle for

Syntaxe

```
for (let compteur = 0; compteur < nbr; compteur = compteur + 1) {  
  /* code source */  
}
```

Boucle for

Exemple

```
for (let compteur = 0; compteur < 3; compteur = compteur + 1) {  
    console.log(compteur)  
}
```

Instruction while

Syntaxe

```
let i = 0
while (i < nbr) {
    /* code */
    i++
}
```

Instruction while

Exemple

```
let i = 0
while (i < 3) {
  console.log(i)
  i++
}
```

https://developer.mozilla.org/fr/docs/Learn/JavaScript/Building_blocks/Looping_code

Les fonctions

Syntaxe

```
function nom_de_la_fonction(paramètre1,paramètre2,...) {  
    /* les instructions*/  
    return s;  
}  
//Appel de la fonction  
let nouvel_variable = nom_de_la_fonction(paramètre1,paramètre2,...)
```

Les fonctions

Exemple

```
function retournerMessageNote(note, nombreQuestions) {  
    let message = 'Votre note est de ' + note + ' sur ' +  
    nombreQuestions  
    return message  
}  
let nouveauMessage = retournerMessageNote(7, 10)  
console.log(nouveauMessage)
```

Récupérer un élément d'une page web

JavaScript a défini plusieurs méthodes de l'objet document (DOM) pour récupérer les éléments de html telles que :

- getElementById;
- getElementsByClassName;
- getElementsByName;
- querySelector;
- querySelectorAll.

https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model

Récupérer un élément d'une page web

Exemple: getElementById

```
<html lang="en">
  <head>
    <title>getElementById example</title>
  </head>
  <body>
    <p id="para">Some text here</p>
    <button onclick="changeColor('blue');">blue</button>
    <button onclick="changeColor('red');">red</button>
  </body>
</html>
```

Récupérer un élément d'une page web

Exemple: getElementById

```
function changeColor(newColor) {  
  const elem = document.getElementById("para");  
  elem.style.color = newColor;  
}
```

<https://developer.mozilla.org/en-US/docs/Web/API/Document/getElementById>

Récupérer un élément d'une page web

Exemple: QuerySelectorAll

```
<body>
  <div class="zoneChoix">
    <input type="radio" name="optionSource"
      id="mots" value="1" checked>
    <label for="mots">Mots</label>
    <input type="radio" name="optionSource"
      id="phrases" value="2">
    <label for="phrases">Phrases</label>
  </div>
  <div class="zoneProposition">
    Entrez le mot : <span>Cachalot</span>
  </div>
</body>
```

Récupérer un élément d'une page web

Exemple: QuerySelectorAll

```
let listeInputRadio = document.querySelectorAll(".zoneChoix input");  
console.log(listeInputRadio);
```

Modifiez un élément d'une page web

setAttribute: permet de modifier les attributs.

```

```

le code javascript suivant permet de modifier la valeur de l'attribut alt

```
let baliseImage = document.getElementById("premiereImage");  
baliseImage.setAttribute("alt", "Ceci est la nouvelle valeur de alt");
```

Modifiez un élément d'une page web

classList: permet de modifier les classes.

```

```

```
let baliseImage = document.getElementById("premiereImage");  
baliseImage.classList.add("nouvelleClasse")  
baliseImage.classList.remove("photo")
```

Ajouter un nouvel élément dans une page web

CreateElement: permet de créer une balise html.

```
let nouvelElement = document.createElement("div");
```

appendChild: permet d'insérer un élément dans la page html.

```
// Récupérer un élément parent existant  
let parentElement = document.getElementById("conteneur");  
// Ajouter le nouvel élément au parent  
parentElement.appendChild(nouvelElement);
```

Événements et interactions

addEventListener: permet d'ajouter un écouteur.

```

```

```
let monBouton = document.getElementById("monBouton");  
monBouton.addEventListener("click", function () {  
  console.log("Vous avez cliqué sur le bouton")  
});
```

Récupérer un élément d'une page web

addEventListener avec une fonction fléchée

```
monBouton.addEventListener("click", () => {  
    console.log("Vous avez cliqué sur le bouton")  
});
```

Bibliographie & Webographie

- JavaScript. L'ESSENTIEL DU CODE ET DES COMMANDES, PEARSON.
- <https://github.com/solygambas/html-css-javascript-projects/tree/main>
- Openclassroom.